COMP 581: Introduction to Robotics

Spring 2025
Instructor: Daniel Szafir (daniel.szafir@cs.unc.edu)
Teaching Assistant: Bryce Ikeda (bikeda@cs.unc.edu)

Lab 2: Wall Following

Deadlines

- Lab pre-flection due **Friday, February 28 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 2 Pre-flection assignment link on Canvas).
- Lab simulation started by **Friday, March 7 at 11:59pm**. Before this "due" time, each team should submit some initial code by committing and pushing it to GitHub; see specific instructions below**.** For the late policy, see the course syllabus.
- Robot lab and final simulation code due **Thursday, March 13 at the beginning of class (12:30 pm)**. Before the due time, each team should submit their final code by committing and pushing to GitHub; see specific instructions below**.** For the late policy, see the course syllabus.
- Lab reflection due **Friday, March 14 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 2 Reflection assignment link on Canvas).

The Objective

The primary purpose of this lab is to implement forward kinematics and wall following for a mobile robot.

Robot Design Requirements

Your robot must be built only using pieces from your Lego Mindstorms kit. You should design your robot such that the dark gray center button may be pressed without moving the robot and such that pushing the button does not disrupt any of the sensors. Your robot should remain in one piece at all times, its diameter with respect to the ground should not exceed 40 cm, and its diameter shall not change substantially during the task. All measurements will be made with respect to the ***measuring point*****:** a distinct point of your choosing that is on the **frontmost surface** of your robot. More specifically, you will place your robot on the ground behind a starting line drawn on the ground, such that your chosen measuring point is **directly above a point on the starting line**, and no part of your robot is permitted to be in front of the starting line. Your robot should be designed to accomplish the task autonomously without any human intervention (except for pushing the dark gray center button as required).

Setup

We have created a Webots simulation library that will enable you to write EV3 MicroPython compatible code in Webots to control a simulated Lego robot in a world that mimics the Lab 2 environment. To download everything you need for this simulation and for deploying your program to the real robot:

1. Log into your GitHub account and then accept the GitHub Classroom assignment by clicking this link: https://classroom.github.com/a/BNoXlFdQ
2. The link will take you to a page where you can accept the assignment on GitHub.
3. After accepting the assignment, it should create the repository for you and your team partner automatically based on the teams created in Lab 1. If you have any difficulty with this, please let us know.
4. The repository will be created and set up in a similar manner as in Lab 1. Please view the repository README for some additional details specific to this lab. You should now be able to edit the `main.py` controller file within the controllers folder in Visual Studio Code and see the updates to the robot's behavior when you run the Webots simulation.
    - Always remember to reset the Webots time back to zero.
5. Your job is now to write the main.py controller to solve the challenge described in the next section below and deploy your solution on your physical robot.

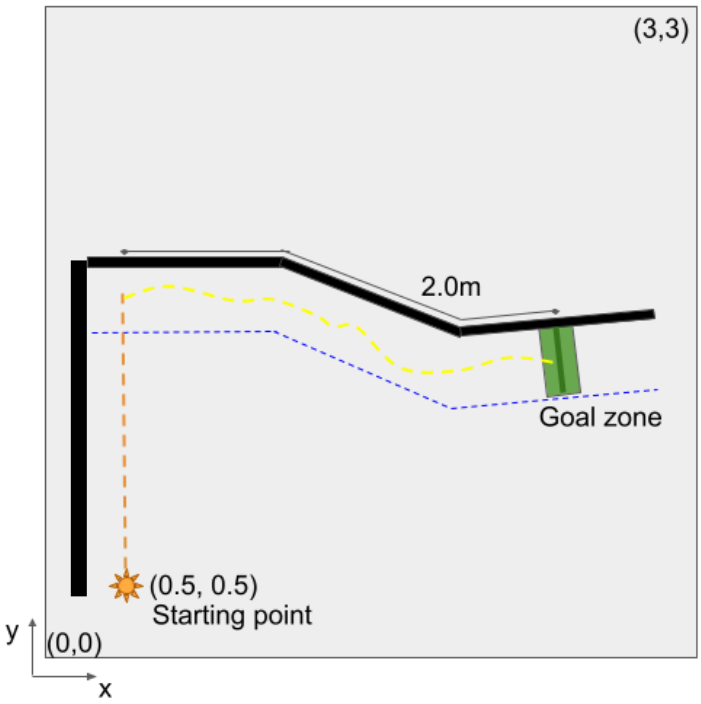
The Task

In this lab, your robot must demonstrate the capacity for autonomous locomotion and wall-following. The task in this lab is to be completed in a single run, without picking up your robot.

Pre-flection
Before programming anything, first read through the task below and make sure you understand it. Then, work with your lab partner (or complete separately and then compare answers) to fill in the Lab 2 Pre-flection worksheet linked in the Lab 2 Pre-flection assignment on Canvas and submit this through GradeScope (only one submission needed per group).

Wall-following Task:

1. You will place your robot behind a starting line such that its **measuring point will be on a specific *starting point*** on the starting line (see figure below).
2. You will then push the dark gray center button. Your robot should then move straight forward (in the positive *y* direction in the figure below, which you may or may not want to consider as your world y axis). Straight ahead, at a distance of somewhere between 60 and 120 cm, will be a wall that is perpendicular to the robot's forward motion. The wall will be at least 17 cm high, of unknown width, and it is bumpable and it can be detected via ultrasound.
3. Once your robot is less than 30 centimeters from the wall, your robot should turn right and follow the wall. It is acceptable to bump the wall and reverse or to detect the wall and stop before hitting it.
4. Once starting to follow the wall, your robot measuring point should never be more than 30 cm from the wall at all times while following the wall (see the blue dotted line in the figure below). The wall may be curved or angled and may bend up to 45 degrees.
5. The robot should follow the wall for a distance of 2.0 m and then stop. The robot should then beep to announce that it has reached its destination.
6. The task (from the time the button is pushed until the robot stops moving) must be completed in under 90 seconds.
7. All of the robot motion must be completed within a 3 meter square workspace without leaving the workspace. In the figure below, the starting point is at coordinate (0.5m, 0.5m) within the full workspace coordinate system.



In the figure above, we illustrate an example of a possible environment that complies with the rules. We note that the environment in your actual run may be different. The robot starts at the starting point. The wall is shown in black. The angles of any wall bend are unknown in advance, but they are at most 45 degrees with respect to the previous wall. *Note that there may be more than one bend*. The dotted blue line represents a 30 cm distance to the wall. The distance is measured along the walls from the point where the robot would ideally reach the wall. The goal zone is perpendicular to the wall. It is 10 cm long and 30 cm wide, thus if your robot stays within 30 cm from the wall and travels along the walls for 2.00m ± 5cm, you will receive full credit. The yellow dashed line shows a possible robot trajectory that would receive full credit if completed in the allotted time, assuming the robot stops inside the green goal zone.

Reflection

After finishing the lab, work with your lab partner to fill in the Lab 2 Reflection worksheet linked in the Lab 2 Reflection assignment on Canvas and submit this through GradeScope (only one submission needed per group).

Grading Rubric

- **(5 Points) Pre-flection:**
    - Effort: 5 points. Points will be awarded for submitting the Pre-flection on time and for making a reasonable effort to answer each question.
- **(45 Points) Simulator:**
    - Initial code upload: 5 points. Points will be awarded if you upload your Webots simulator code that appears to have a reasonable effort in (e.g., the robot moves, detects the wall and tries to turn right) by the simulator deadline (March 7).
    - Correct code: 40 points. Points will be awarded if you upload your code which successfully accomplishes the tasks in the simulator by the full lab deadline (March 13). The division of points across simulator tasks is identical to that described below for the real robot attempt.
- **(45 Points) Real Robot Attempt:**
    - Effort: 5 points. You will receive these points by:
        - Showing up to the lab <u>on time:</u> 2 points
        - Having your robot built and ready to attempt the course: 1 point
        - Pointing out your robot's measuring point according to the definition given in this document: 1 point
    - Robot Task: 40 points, with a breakdown as follows:
        - Detect the wall: 5 points. You will receive these points if your robot stops and attempts to turn right when it approaches the wall (or bumps the wall, backs up an appropriate distance, and attempts to turn right).
        - Turn at the wall: 10 points. You will receive these points if your robot successfully turns right and starts following the wall.
        - Following the wall: 10 points. You will receive these points if your robot is never more than the allowable maximum distance from the wall. This requirement must be satisfied 40 cm after the robot is supposed to turn right until the destination (i.e., you have a 40cm "buffer zone" to get your robot the correct distance to the wall). Partial credit will be awarded.
        - Accuracy of reaching goal: 15 points. Points will be based on the accuracy of reaching the robot's target location when it stops at the end of the task. Full points will be awarded if the robot stops inside the goal zone ± 5cm where it ideally should end. Partial credit will be given based on distance to this goal zone.
- **(5 Points) Reflection:**
    - Effort: 5 points. Points will be awarded for submitting the Reflection on time and for making a reasonable effort to answer each question.

Penalties:

- There will be a -10 point penalty if, after crossing the start line, your robot ever exits the 3m by 3m workspace.
- There will be a -10 point penalty if more than 90 seconds elapse from when the dark gray

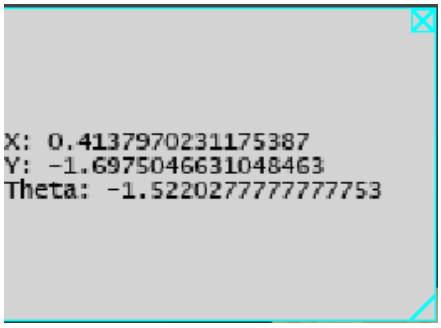
center button is pushed at the start location until the robot stops.
- If the robot is still moving after 2 minutes, then the instructor will stop the run and no additional points can be earned.

Extra Credit:

This lab has two extra credit opportunities:

1. **Style:** You may earn up to 5 bonus point for personalizing your robot. This can be anything from sound effects to additional Lego parts, but it must represent your team's personality. Customizations that are fun and creative will earn more points.

2. **Debugging:** To help with your debugging efforts, you may earn up to 5 bonus points by continuously printing your robot's x, y, and theta values on the screen. This can help to determine if you are having any odometry issues as your robot moves. You may also print the distance it thinks it has traveled. This can be done in both Webots and on the physical robot. If you cannot see the display in Webots, click Overlays -> "Hide All Display Overlays." (Note: The Webots simulator renders text in the vertical direction farther than on the real robot). To earn these points, show us the robot printing out these values during your run. An example is below:



Code

To complete this lab, you will use a subset of the EV3 MicroPython API. You may only use the following modules:
ev3brick
ev3devices
parameters
tools
media
You may *not* import, use, or copy the "robotics" module, or any of its included classes or functions, in your program.

A robot relying on a module not allowed as stated above will be disqualified. Also, any robot using Wi-Fi or Bluetooth communications in any way will be disqualified.

You should submit a zip file of your Python source code files on GradeScope. The zip file should include all the Python source code files you wrote (which should have a .py file extension). In the comments at the top of the Python source file containing your main .py file, please include the names and PIDs of all team members.

You are free to discuss concepts in the course with other students or AI tools (e.g., ChatGPT) in natural language, but you should not share any programming code with others and you must write all the robot's programming code yourself. If you access any sources other than the textbook or course material posted on Canvas, you must cite them in comments at the top of your code. You must fully understand your code and be able to reproduce the algorithmic approach without references if asked. The Honor Code is in effect for these policies.

Good Luck and Have Fun!